

## ACPL schematic $\rightarrow$ repo files (block-by-block)

### Input episode (white box on the left)

- **Initial state  $\psi_0$  (arc basis)**  
acpl/sim/state.py: `init_arc_state(G, portmap, kind="localized" | "superpos" | ...)`.  
acpl/sim/portmap.py: defines arc/port indexing used by the state layout.
- **Horizon  $T$ , time feature  $t, t/T$**   
acpl/train/loops.py: rollout loop that iterates `t = 0..T-1`.  
acpl/policy/policy.py: assembles the time feature (e.g., `t/T`, sinusoidal time PE) that's concatenated into the controller input.
- **Graph  $G = (V, E)$  (edge\_index, degrees, port map)**  
acpl/data/graphs.py: generators for line/grid/hypercube/ER/WS/d-regular graphs.  
acpl/sim/portmap.py: builds the **port map** (flip-flop partner at each vertex) used by Shift  $S$ .  
acpl/data/splits.py, acpl/data/generator.py: on-the-fly seeded episode graphs.
- **Node features  $X$  (degree, LapPE, coords/bitstrings, roles)**  
acpl/data/features.py: degree vectors, Laplacian positional encodings (with sign disambiguation), coordinates, bitstring labels, optional role encodings.  
acpl/policy/pe.py: toggles/merges positional encodings into the model input.
- **Task spec  $f(y, T)$  (transfer / search / mixing / robust)**  
acpl/configs/task/\*.yaml: task family + parameters (targets, marked sets, disorder levels...)  
acpl/objectives/{transfer,search,mixing,robust}.py: the objective definitions used to score  $\psi_T$ .

### Policy $\pi_\omega$ (blue box)

- **GNN encoder  $\phi_{\text{GNN}}$  (message passing)  $\rightarrow$  node embeddings  $z_v$**   
acpl/policy/gnn/{gcn,graphsage,gat,gin,pna}.py: concrete encoders.  
Each exposes something like `forward(G, X) -> z` with shape `[|V|, d_z]`.
- **Node embeddings  $\{z_v\}$**   
**Where produced:** by the selected GNN encoder.  
*Files:* acpl/policy/gnn/{gcn,graphsage,gat,gin,pna}.py  
*Output:* `z = encoder(G, X)` with shape `[|V|, d_z]`.  
**Where used:** fed (per time-step) into the temporal controller.  
*File:* acpl/policy/policy.py (policy assembly)  
*Detail:* the controller input is typically `cat([ z_v, time_feat(t/T), pos_enc(v) ])`.
- **Positional encodings (graph PE / coords / bitstrings)**  
*Where defined/constructed:* acpl/data/features.py (Laplacian PEs with sign disambiguation, geometric coords, bitstrings, roles) and acpl/policy/pe.py (routes/normalizes PEs and controls which PE variant is appended).  
*Where injected:* acpl/policy/policy.py concatenates PE alongside  $z_v$  and time features before the controller. Shapes: `pos_enc` is `[|V|, d_pe]`; controller sees `[|V|, d_z + d_pe + d_time]`.

- **Temporal controller (GRU / Transformer)**  
`acpl/policy/controller/gru.py`: `TemporalGRU(d_in, d_h)`; consumes `[z_v ; time features ; optional PE]`.  
`acpl/policy/controller/tiny_transformer.py`: `TemporalTransformer(d_in, n_layers, n_heads, ...)` for attention-based control.
- **Coin parameters  $\{\theta_{v,t}\}$  (explicit “coin params” box)**  
`acpl/policy/head.py`: `CoinParamHead(d_h, param_spec)`; projects controller states to the **exact parameter tensor** per vertex/time. For  $d = 2$  (SU(2)) heads: outputs 3 angles (e.g., ZYZ Euler) or an axis-angle triple. For  $d > 2$ : outputs a skew-Hermitian generator  $K$  in a compact parameterization.
- **Unitary lift  $\Phi(\theta)$  (the “unitary lift” box)**  
`acpl/sim/coins.py`:  
 $d = 2$ : `su2_from_euler(theta)` or `su2_from_axis_angle(theta)`  $\Rightarrow C_v(t) \in \text{SU}(2)$ .  
 $d > 2$ : `coins_exp_from_skewK(K)` (matrix exponential) and `coins_cayley_from_skewK(K)` (Cayley retraction)  $\Rightarrow C_v(t) \in \text{U}(d)$ .  
Both paths implement  $C_v(t) = \Phi(\theta_{v,t})$ .
- **Local coins  $\{C_v(t)\}$  and block-diagonal assembly  $C_t = \bigoplus_v C_v(t)$**   
`acpl/sim/coins.py`: `build_local_coins(theta, degrees, family=...)` returns all  $C_v(t)$ .  
`acpl/sim/utils.py`: `block_diag_coins(C_v_list, layout)` builds the **block-diag** operator  $C_t$  aligned with the arc basis.

## DTQW Simulator (green box)

- **Shift  $S$  (flip-flop permutation on arcs)**  
`acpl/sim/portmap.py`: `flipflop_pairs(G)` computes arc partner indices.  
`acpl/sim/shift.py`: `build_shift_perm(portmap)` returns the permutation/unitary  $S$ ; `apply_shift(S, psi)` applies it.
- **Step  $U_t = S \cdot C_t$  (the “Step  $U_t$ ” node)**  
`acpl/sim/step.py`: `build_step(S, C_t)` and `apply_step(psi_t, C_t, S)` implement
$$\psi_{t+1} = S(C_t \psi_t)$$
in arc basis with the block-diag coin.
- **Evolve  $\psi_{t+1} = U_t \psi_t$  for  $t = 0..T - 1$**   
`acpl/sim/step.py`: one-step primitive (used inside training/eval loops).  
`acpl/train/backprop.py`: differentiable rollout unrolling  $t = 0..T - 1$ , calling `apply_step` each time.  
`acpl/train/ppo.py`: rollout for policy-gradient objectives.
- **Reduced position  $\rho_T = \text{Tr}_C(|\psi_T\rangle\langle\psi_T|)$**   
`acpl/sim/utils.py`: `partial_trace_coin(psi_T, layout)` marginalizes the coin DOF to position space.
- **Position probabilities  $P_T = \text{diag}(\rho_T)$**   
`acpl/sim/utils.py`: `position_probs_from_state(psi_T)` (or via `partial_trace_coin + diag` extraction).  
`acpl/objectives/metrics.py`: helpers to check normalization / compute distances on  $P_T$ .

## Learning & Evaluation (yellow box)

- **Loss  $L = f(\psi_T)$  (transfer/search/mixing/robust)**  
acpl/objectives/transfer.py: maximize  $P_T$  at a target node (state transfer).  
acpl/objectives/search.py: marked-node search objectives.  
acpl/objectives/mixing.py: minimize TV/JS/KL to uniform or to a reference distribution.  
acpl/objectives/robust.py: robustness (e.g., disorder expectations, CVaR).
- **Backprop through time (or PPO/A2C)**  
acpl/train/backprop.py: BPTT, gradient clipping, EMA, checkpointing.  
acpl/train/ppo.py: policy-gradient variants for non-diff or sampled rewards.  
acpl/train/loops.py: epoch/episode orchestration, logging hooks.
- **Validation / Test — Baselines · Ablations · CIs**  
acpl/eval/protocol.py: deterministic CI evaluation over seeds/splits; aggregates mean  $\pm$  CI for metrics (H, TV, JS, KL-pu, Hellinger, L2).  
acpl/eval/ablations.py: toggles like No-PE / Global-Coin / Time-Frozen / Node-Permute.  
acpl/eval/plots.py:  $P_t$  curves, mixing traces, robustness sweeps.  
scripts/eval.py: CLI entry for saved checkpoints.  
scripts/viz\_coins.py, scripts/export\_coins.py: visualize or dump learned  $\{C_v(t)\}$ .

## Configs & entrypoints (wiring the whole path)

- acpl/configs/model/\*.yaml: choose **GNN, controller, coin family** (SU(2)/exp/Cayley), hidden sizes, heads, etc.
- acpl/configs/train.yaml, acpl/configs/optim.yaml: episodes per epoch,  $T$ , LR, schedulers, grad-clip, EMA.
- acpl/configs/task/\*.yaml: task flavor + its target/marked set/mixture reference.
- scripts/train.py: the end-to-end CLI that instantiates:
  1. **Input episode** (graph + features +  $\psi_0$ ),
  2. **Policy**  $\pi_\omega$  to produce  $\{\theta_{v,t}\} \rightarrow \{C_v(t)\}$ ,
  3. **Simulator** (coins  $\rightarrow$  step  $U_t \rightarrow \psi_T \rightarrow P_T$ ),
  4. **Loss + optimizer**, and the train/val loops.

## Quick “who does what” index for the callouts

- **Coin params  $\{\theta_{v,t}\}$** : acpl/policy/head.py (produced), fed to acpl/sim/coins.py.
- **Unitary lift  $\Phi(\theta)$** : acpl/sim/coins.py (su2\_from\_euler, coins\_exp\_from\_skewK, coins\_cayley\_from\_skewK).
- **Local coins  $\{C_v(t)\}$  + block-diag  $C_t$** : acpl/sim/coins.py (per-node) + acpl/sim/utis.py (block-diag assembly).
- **Shift  $S$** : acpl/sim/portmap.py (pairs) + acpl/sim/shift.py (perm/unitary + apply).
- **Step  $U_t = S \cdot C_t$** : acpl/sim/step.py (apply\_step / build\_step).

- **Evolve**  $\psi_{t+1} = U_t \psi_t$ : `acpl/train/backprop.py` (unrolled rollout) using `apply_step`.
- **Reduced position**  $\rho_T$ : `acpl/sim/utils.py` (`partial_trace_coin`).
- **Position probs**  $P_T$ : `acpl/sim/utils.py` (`position_probs_from_state`).